Performance Engineering of a PySpark Pipeline on NYC Yellow Taxi Trips

Tomás Neves, 72901
Hugo Feteira, 73658
tmb.neves@campus.fct.unl.pt
h.feteira@campus.fct.unl.pt

ABSTRACT

We built a PySpark pipeline that cleans and analyses one year of New York yellow-taxi trips (84.6 million records) and optimised it step by step, starting every change from a measurement. On an Apple M4 laptop running a 3-worker Docker cluster, the pipeline went from 139 s to 99 s (−29 %). The same changes on an older Intel laptop gave −59 %. The biggest win came from a surprise in the input files: each one is stored as a single block, so Spark read each file with only one task. Other changes came from reading the execution plans. One idea, keeping all trips in memory, crashed the executors, and we explain why. Comparing the two machines shows that the bottleneck moves: on the slow CPU the work dominates, while on the fast one writing to Docker's shared folder does, and adding workers on one laptop makes the pipeline slower.

KEYWORDS

Apache Spark, performance engineering, data skew, Parquet, caching

1 INTRODUCTION

The New York Taxi and Limousine Commission publishes every yellow-taxi trip as one Parquet file per month [2]. We use the full year 2019: 12 files, 1.2 GB and 84.6 million trips. It is the last full year before the 2020 drop in traffic, all its files share the same column types, and it is small enough to repeat every measurement several times on our machines. On top of this data we implemented five operations:

- **M1, cleaning:** remove invalid trips and count how many each rule removes.
- **M2, temporal profile:** trips, mean fare, duration and distance per borough, weekday and hour.
- **M3, zone ranking:** monthly ranking of pickup zones, a rolling 3-month share, and alerts for unusual volume.
- **O2, flows:** trips between every pair of zones, the 20 busiest routes per borough, and routes whose share goes up in one quarter and down in another.
- **O3, demand features:** for each zone, weekday and hour, the historical mean and variance, the value 4 weeks earlier, and a high-demand flag.

We chose O2 and O3 because each one stresses a different part of Spark. **O2** groups by every pair of zones (up to 263^2 keys per month) and joins the zone table twice, so it exposes *shuffle volume* and *join ordering*: joining before or after the aggregation changes how much data moves. Its windows are partitioned by borough, and the keys are skewed (Manhattan has about 92 % of the pickups). **O3** models the 4-week lag as a *self-join*, and uses the same hourly counts three times (mean, lag and the lag join), so it exposes the cost of two extra shuffles and of recomputing a reused intermediate result. Both are interesting targets because they are the most expensive steps (62 % of the baseline run) and their costs come from choices made in our own code, such as the join order and what is reused, so we can change them and measure the effect. Both also start from the same cleaned trips as M1–M3, so how that data is stored and reused matters more than the arithmetic.

2 PIPELINE

`src/main.py` runs M1 first and then each operation as a separate module. All modules share the same signature, `run(trips, zones, args, opt)`. Each operation runs inside the profiler from Lab 3, which saves its metrics and its execution plan (`explain("extended")`) to `results/`. Every optimisation is a flag (`--opt name`). Baseline and optimised versions are therefore the same code, and we can always compare them.

A few design decisions shaped the code:

- **One pass for the cleaning statistics.** M1 evaluates its six cleaning rules inside a single aggregation instead of six `filter().count()` calls, so the data is read once, not six times. The counts are saved per year (`m1_stats`). For 2019 it removes 3.80 million of 84.6 million trips (4.5 %): passenger count 0 or above 6, 1.53 M; zone outside 1–263, 1.19 M; distance ≤ 0, 0.75 M; nulls in a used column, 0.44 M; fare ≤ 0, 0.21 M; dropoff not after pickup, 0.08 M. A trip can fail several rules, so the counts add up to more than the total.
- **Months with no trips still exist.** M3 builds the full grid of zones × months before computing rolling windows, so a month with zero trips counts as zero instead of being skipped. O3 does the same with hours: it divides by the number of calendar days, not by the number of rows.
- **No alerts in empty zones.** In zones with almost no trips the standard deviation is close to zero, so any change looked like an anomaly. We only raise alerts when the zone averages at least 30 trips per month (M3) or 1 trip per hour (O3). M3 also compares each month with the three months *before* it. With the current month included in the window, the alert could never fire.
- **How O2 reads the assignment.** The top-20 is computed over the whole loaded period, since no period is given. A quarter's direction is the sign of the share's change from its first to its last month (Jan→Mar, Jul→Sep), and a route reverses when the two signs differ. We test this on the top-20 routes only: on small routes the share swings by chance and almost all of them would "reverse".
- **DataFrame API everywhere.** It produces the same Catalyst plans as SQL, and it works naturally with our Python helpers: the dictionary of cleaning rules and the `opt` flags.

Figure 1 shows the plan of the main O2 query. We refer to it in Section 4.

```
SortMergeJoin [pickup, dropoff]          <- top-20 x shares
+- Exchange hashpartitioning(pickup, dropoff)
   +- Filter route_rank <= 20
      +- Window row_number (borough)      <- top-20 per borough
         +- Exchange hashpartitioning(borough)
            +- HashAggregate sum(trips) per route
               +- Exchange hashpartitioning(route)
                  +- BroadcastHashJoin <- zone names
                     +- BroadcastHashJoin <- zone names
                        +- HashAggregate count per route
                           +- Exchange hashpartitioning
                              +- FileScan parquet m1_trips [4 columns]
```

Figure 1: Simplified physical plan of O2 after OPT-2. The trips are counted per route *before* the joins to the zone names. Until OPT-4, Spark ran this whole subtree again every time the result was used.

3 PROFILING AND BASELINE

Setup. The main measurements ran on a MacBook with an Apple M4 (10 cores) and 16 GB of RAM. Docker had 10 virtual CPUs and 8.2 GB. The Spark 4.2.0 standalone cluster had 1 master and 3 workers, each with 2 cores and 1500 MB (executors 1200 MB, driver 1 GB). The data sits in a folder shared between macOS and the containers (a Docker bind mount). We kept Spark's defaults: adaptive query execution (AQE) on, 200 shuffle partitions. The exact `spark-submit` commands, and one script that reruns every measurement in this report, are in the repository's `README.md`. Each number is the median of 3 runs on the full year, and no other containers ran during the measurements. We first measured on an older laptop (Intel i7-6920HQ, 4 cores, Docker with 6 virtual CPUs and 8 GB) and compare the two machines in Section 5.1.

A word on noise. Identical runs of the baseline took 127, 139 and 173 s, and the whole third round was slower. M1, the step that writes to the shared folder, is the noisiest: with the same M1 code it took 56 s in one run and 126 s in another of the same round. For each comparison we therefore also report metrics that do not depend on the machine's load (input records, number of tasks, shuffle bytes), and we interleaved the runs (A, B, C, A, B, C, …) so that every version faced the same conditions.

Table 1: Baseline (Apple M4, 2019, median of 3 runs).

	M1	M2	M3	O2	O3	total
time (s)	16	15	16	46	40	139
tasks	157	146	198	840	559	
records read (M)	84.6	84.6	84.6	197.6	169.2	
shuffle read (MB)	0.0	0.7	1.5	79.3	66.0	
shuffle write (MB)	0.0	0.7	1.5	69.5	50.3	
slowest stage (s)	10.5	13.9	13.8	22.5	35.9	
its skew (max/med.)	51	64	86	159	220	

The baseline revealed three problems:

(1) **One task does almost all the reading.** In every operation the slowest stage is the read stage, with 72 tasks for 12 files. Its slowest task is 51 to 220 times longer than the median one (Table 1), far above the 10× that signals skew. In O3, for example, one task took 10 s and the median 0.05 s. Checking `spark_partition_id()` showed why: all rows of a file ended up in the same partition. Every TLC file is written as a single Parquet row group [3]. Spark splits each file into 3–6 pieces, but only the piece that holds the row group has data. In practice Spark reads with one task per file.

(2) **The same work is done again and again.** No operation reuses another's result, so each one reads and cleans the raw files from scratch. The O2 plan scans the 12 files 3 times, plus the 4 quarter months a fourth time, and O3's 3 times. O2 and O3 read more than twice as many records as the dataset has (Table 1), and together they take 62 % of the run.

(3) **O2 joins too early.** O2 attaches the zone names to every one of the 80 million trips before counting them, and then shuffles rows that carry four text columns.

The other plan checks. The `FileScan` reads only the 9 columns that the cleaning rules use, not the whole file, so column pruning already works. The shuffles are small (at most 79 MB), and most of each stage's time goes to the scan that feeds the `Exchange`, not to the `Exchange` itself. The zone table is always joined with a `BroadcastHashJoin`. The `SortMergeJoins` (3 in O2, 3 in O3) join aggregated results, not trips. In O3 both sides of the lag self-join have about 1 million rows, too many to broadcast. None of them is a bottleneck.

4 OPTIMISATION STUDY

4.1 OPT-1: clean once, store in smaller blocks

What we saw: problems 1 and 2. Reads were limited to one task per file, and every operation cleaned the raw data again.

What we expected: if we write the cleaned trips once, with smaller row groups (32 MB), every later read can use several tasks per file and the cleaning runs only once.

What we changed: `--opt materialize`. M1 writes the clean trips to disk, and M2–O3 read from there.

What happened: the pipeline dropped from 139 s to 110 s (−21 %). M2–O3 together went from 117 s to 56 s (−52 %), and O2 from 840 to 242 tasks. But M1 grew from 16 s to 54 s, because it now writes 1.5 GB to the shared folder. That write eats more than half of the gain. The results are identical to the baseline.[1]

4.2 OPT-2: count first, add names later

What we saw: problem 3. In the O2 plan, both joins with the zone table processed every trip.

What we expected: counting trips per route (pickup zone, dropoff zone, month) first leaves at most 263^2 rows per month to join. Catalyst does not move a `groupBy` below a join by itself, so we had to do it in the code.

What we changed: `--opt agg_first` (Figure 1).

What happened: O2 went from 21.4 s to 9.7 s (−55 %). Executor CPU time fell from 93 s to 31 s, and the shuffled data from 75 MB to 39 MB. The pipeline now took 99 s, −29 % against the baseline.

[1] Checked with `exceptAll` in both directions. One of the 993 M2 rows differs in the 14th decimal place, because floating-point sums depend on the order of the partitions.

4.3 OPT-3: keep all trips in memory (rejected)

What we saw: OPT-1's profile. The write to the shared folder made M1 grow from 16 s to 54 s, more than half of what OPT-1 saved.
What we expected: `persist(MEMORY_AND_DISK)` [5] on the cleaned trips would avoid the repeated reads without paying for OPT-1's write.
What we changed: `--opt cache`, instead of `--opt materialize`.
What happened: the job failed on both machines. The executors ran out of memory (`OutOfMemoryError: Java heap space`), each task failed four times, and Spark aborted the job.
Why: each executor has 1200 MB, shared between cached data and running tasks. Once decoded in memory, the trips take several times their 1.5 GB on disk. The cache took the memory that the joins and aggregations needed. OPT-1 gives the same reuse without using executor memory.

4.4 OPT-4: cache only the small results

What we saw: even after OPT-1 and OPT-2, the O2 plan still scanned the trips 4 times and O3 3 times. The intermediate results (route counts in O2, hourly counts in O3) are used several times, and Spark recomputed them every time.
What we expected: these results are small (330 thousand and 1 million rows), so caching them is safe, unlike in OPT-3. Each operation would read the trips only once.
What we changed: `--opt cache_agg`.
What happened: the reads went down as expected: O2 now reads 80.8 million records instead of 188.8 million. But O2 became **almost twice as slow** (14 s to 26 s, Table 2), and its task count jumped from 226 to 2983.
Why: in the plan, every shuffle is followed by an `AQEShuffleRead coalesced` step, which merges small partitions. The last shuffle before the cache is not. AQE [4] keeps the partitioning of a cached plan, so the cache holds 200 tiny partitions. Each of its 4 reads then starts 200 tasks, and each task has a fixed start-up cost.

4.5 OPT-5: fewer shuffle partitions

What we saw: the 200 tiny partitions left by OPT-4.
What we expected: the default of 200 partitions is meant for large clusters [1, 4]. We have 6 cores and a few MB of aggregated data. With 24 partitions (4 per core) the cached data would be split sensibly, and AQE would still merge what remains.
What we changed: `spark.sql.shuffle.partitions=24`, a configuration setting with no code change.
What happened: together with OPT-4, O2 and O3 went from 31 s to 25 s (−19 %), and O2 from 2983 to 513 tasks. The gain is in O3 (16 s to 11 s, CPU 73 s to 42 s), where the reused hourly counts are expensive to recompute. O2 is back to its OPT-2 time. As a control, OPT-5 *without* OPT-4 (row D) gives 27 s, and the small gain is in O2: without the cache, AQE was already merging those partitions. OPT-4 removes the repeated reads, and OPT-5 removes the cost that OPT-4 added.

5 ANALYSIS

We ran the final version (OPT-1, 2, 4 and 5) with 1, 2 and 3 workers on the full year, and with 10 % and 20 % of the data on 3 workers, in 3 interleaved rounds. We did not use the DI cluster: as the course allowed, our cluster is the multi-node Docker setup described above, with each worker in its own container.

Table 2: OPT-4 and OPT-5, interleaved runs (Apple M4, median of 3).

version	O2 (s)	O2 tasks	O3 (s)	O2+O3 (s)
A: OPT-1 + OPT-2	14	226	16	31
B: A + OPT-4	26	2983	19	44
C: A + OPT-4 + OPT-5	14	513	**11**	**25**
D: A + OPT-5	12	219	15	27

Table 3: Summary of all optimisations (Apple M4, 2019, median of 3 runs).

version	total (s)	effect
baseline	139	
+ OPT-1 clean once	110	−21 % (M2–O3 −52 %)
+ OPT-2 count first	99	−29 %; O2 −55 %
OPT-3 cache all trips	fails	out of memory, rejected
+ OPT-4 cache small results	–	O2+O3 +45 % alone
+ OPT-5 24 partitions	−*	O2+O3 −19 % with OPT-4

*In full-pipeline runs this 6 s gain is hidden by the noise of the full runs (91 to 248 s).
The fair comparison of OPT-4 and OPT-5 is the interleaved one in Table 2.

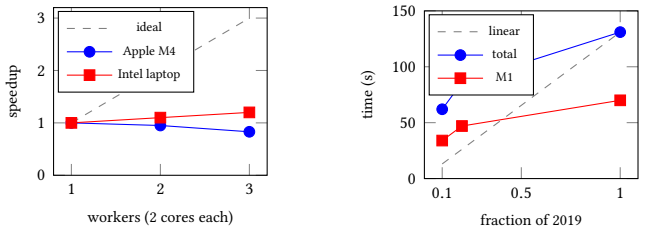


Figure 2: Left: speedup over 1 worker. On the M4: 109 s, 115 s and 131 s for 1, 2 and 3 workers. Right (M4, 3 workers): run time for 10 %, 20 % and 100 % of the data.

More workers make it slower. On the M4, 2 workers take 1.05 times as long as one and 3 workers 1.2 times (speedup 0.95 and 0.83, efficiency S/p of 0.48 and 0.28; on the Intel laptop 0.55 and 0.40). The pattern repeated in all 3 rounds, and the logs confirm 1, 2 and 3 executors. Spark does use the extra workers: in M1, the number of tasks running at the same time goes from 1.8 to 3.5 to 5.0 on average. But M1 stays at 68–70 s, because its time goes into reading and writing the shared folder, and that does not get faster with more workers. In M2–O3 the total CPU time grows from 55 s to 175 s for exactly the same work. With one executor, the shuffles stay inside one JVM. With three, each shuffle crosses containers and each step pays coordination costs that are large next to 10 s of real work. On a real cluster each worker would bring its own disk and network, so this is a limit of our single-machine setup.

More data costs less than proportionally. Ten times more data takes 2.1 times longer (62 s to 131 s). There are three reasons:

- the sample is taken *after* reading, so M1 always reads all 84.6 million records;
- O2 and O3 run 18–43 stages and 280–510 tasks regardless of input size, and at 8 million rows this fixed cost dominates;
- the results are aggregates whose size is bounded by the number of zones, days and hours, so everything after the first aggregation hardly shrinks.

5.1 Two machines, two bottlenecks

Table 4 compares the M4 with the older Intel laptop, using the same code and cluster topology.

Table 4: The same pipeline on two machines (median of 3 runs; Intel scaling: median of the 2–3 valid runs).

	Intel i7 (4 cores)	Apple M4 (10 cores)
baseline (s)	1398	139
OPT-1 (s)	661 (−53 %)	110 (−21 %)
OPT-1+2 (s)	567 (−59 %)	99 (−29 %)
O2+O3, A → C (s)	117 → 76 (−35 %)	31 → 25 (−19 %)
speedup, 3 workers	1.20	0.83
time for 10× the data	1.5×[†]	2.1×

[†] 2 runs per fraction; on the Intel laptop 20 % ran faster than 10 %, so this ratio is within the noise. One 1-worker run (37,761 s; the laptop went to sleep) was discarded.

The baseline is 10 times faster on the M4, but the optimisations gain less. On the Intel laptop the work itself (decoding, cleaning, joining) was the bottleneck, so removing repeated work paid off fully. On the M4 that work is cheap, and what remains is moving data through the shared folder: OPT-1's write costs 38 s there, more than a quarter of the baseline. The same goes for scaling. On the Intel laptop the 3 workers shared 4 physical cores and still gained a little (1.20). On the M4 there are enough cores, so the shared disk and the coordination between containers decide, and adding workers hurts. Which optimisation is worth it depends on where the bottleneck is on that machine.

6 DISCUSSION

What is still slow. M1 is now more than half of the run (56 of 99 s). It reads the raw files twice (once for the statistics, once to write the clean copy), and each file is still read by one task. The 4-week lag in O3 is a self-join, which needs a shuffle and a sort-merge join. A `lag` window over the date would avoid it. The plans of O2 and O3 also show sort-merge joins between small tables. We did not check whether AQE turns them into broadcast joins at run time.

Limits of our measurements. Both machines are laptops, and the cluster nodes are containers on the same machine. Identical runs varied a lot (the OPT-1 version took 107 and 220 s), mostly in M1. We dealt with this using medians, interleaved runs and load-independent metrics, but gains of a few seconds stay below the noise in full-pipeline runs. Our speedup numbers describe one machine, not the scaling of a real cluster.

With more time, we would run the full 2016–2020 range (five times more data) on a real cluster, where each worker has its own disk. We would also compute the M1 statistics and the clean copy in a single pass, keep the clean copy on the containers' local disk instead of the shared folder, sample by file instead of after reading, and tune AQE's partition size and skew-join thresholds, since a few Manhattan zones dominate the keys.

ACKNOWLEDGEMENTS AND AI USAGE

We used Claude Code (Anthropic, Claude Opus model) as a tutor and pair programmer. The full disclosure, with every prompt quoted from the session transcripts, is in `docs/ai-usage.md`.

Did we build an agent? No. We used Claude Code as it is, with a few fixed rules. Tomás's rules: the AI gives the code in small blocks and explains every line, and he runs each block himself; the AI follows the lecture and lab material; it never commits under its own name and never commits the data. Hugo's rules: a global instruction file (no AI signature; helper agents work in separate copies of the repository and never merge or push).

Development. The AI helped plan the project from the assignment and set up the Docker cluster. With Tomás it wrote M1, M2 and O2 block by block, and with Hugo it wrote the M3, O2 and O3 notebooks and tested them on small synthetic data. An AI-assisted audit of Hugo's notebooks found two bugs in O2, which Hugo then fixed. Example prompts: "give me the code, I will do it, but explain everything and the reason for each thing"; "fetch Hugo's commits and do a full audit". Design decisions were ours, for example the period of the top-20 ranking and the minimum volume for alerts. The AI also generated the helper scripts (`run.sh`, `bench_all.sh`, `scaling.sh`, `compare.py`, `summarize.py`) and drafted the text of this report from our notes and results; we reviewed and corrected both.

Optimisation. The AI ran the benchmarks (`scripts/run.sh`, `bench_all.sh`), read the plans and metrics with us, implemented each optimisation behind an `opt` flag, and checked that the results stayed identical to the baseline. The analysis behind each optimisation is in Section 4: OPT-1 came from the skewed read stage and the partition IDs, OPT-2 from the O2 plan, OPT-3 from the out-of-memory log, OPT-4 from the scan counts, and OPT-5 from the missing `AQEShuffleRead` in the cached plan. The AI also pointed out the measurement noise, suggested interleaving the runs, and suggested repeating all measurements on the faster machine. Example prompts: "do you want to test it and see if we can optimise something?"; "more optimisation strategies for the report?". Every number in this report comes from our own profiler output in `results/` and `results-hugo/`.

INDIVIDUAL CONTRIBUTIONS

Tomás Neves, 72901 (50 %): Docker cluster and data; M1 and M2; the profiler and `main.py`; audit of M3, O2 and O3 and their integration into the pipeline; all optimisations and the first round of measurements (`run.sh`, `scaling.sh`, `bench_all.sh`, `compare.py`); the report.

Hugo Feteira, 73658 (50 %): M3 (module and step-by-step notebook); O2 and O3 notebooks (design and logic, later integrated by Tomás); fix of the two O2 bugs found in the audit; synthetic tests and answers to the audit; the final measurements on the Apple M4; review of the report (M3, O3) and his part of the AI disclosure.

COMMENTS AND SUGGESTIONS

The step from "it runs" to "we know why it is slow" was the most valuable part of the project. Most of our improvements came from reading the plans and the metrics, not from changing settings. Measuring on two machines also taught us that a bottleneck belongs to a machine, not to the code alone.

REFERENCES

[1] Amazon Web Services. 2026. Using Adaptive Query Execution (Spark tuning for AWS Glue and Amazon EMR). https://docs.aws.amazon.com/prescriptive-guidance/latest/spark-tuning-glue-emr/using-adaptive-query-execution.html.
[2] NYC Taxi and Limousine Commission. 2026. TLC Trip Record Data. https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page.
[3] The Apache Software Foundation. 2026. Apache Parquet File Format: Row Groups. https://parquet.apache.org/docs/file-format/.
[4] The Apache Software Foundation. 2026. Spark SQL Performance Tuning: Adaptive Query Execution. https://spark.apache.org/docs/latest/sql-performance-tuning.html.
[5] Matei Zaharia, Mosharaf Chowdhury, Tathagata Das, Ankur Dave, Justin Ma, Murphy McCauly, Michael J. Franklin, Scott Shenker, and Ion Stoica. 2012. Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing. In *Proc. 9th USENIX NSDI.*